PRÁCTICA 10

Empaquetar la aplicación del proyecto en un contenedor

Equipo	Los Rojos
Materia	Cómputo en la Nube · Grupo 503
Unidad	Unidad II · Segundo parcial
Sesión	Jueves 29 de octubre de 2026
Integrantes	• Flores Guevara Ángel Gabriel • Kevin Ricardo López Payán
Docente	Elizabeth Gaxiola Carrillo
Objetivo	Construir la imagen de contenedor de la aplicación del proyecto y publicarla en un registro privado.

1. Imagen base oficial y justificación (C1 — 15 pts)

Se eligió node:20-alpine como imagen base. La siguiente tabla compara las alternativas consideradas:

Imagen	Tamaño comprimido	Ventaja	Decisión
✓ ELEGIDA node:20-alpine	~55 MB	Alpine Linux (~5 MB). Imagen oficial. Node 20 LTS (soporte activo). Incluye las herramientas mínimas necesarias (npm, node).	USADA
✗ Descartada node:20	~350 MB	Debian completo — herramientas de compilación y shells que no se usan en producción.	Muy pesada
✗ Descartada ubuntu:22.04 + node manual	~180 MB	Imagen genérica no oficial para Node. Requiere instalar Node manualmente con pasos adicionales.	No oficial para Node

1.1 Dockerfile completo comentado línea por línea

Cada instrucción está comentada explicando su propósito, la justificación del orden y las decisiones de seguridad:

```
# Imagen base: node:20-alpine
# Alpine Linux (~5 MB) + Node 20 LTS — oficial en Docker Hub, auditada.
# Mucho más ligera que node:20-debian (~350 MB) o ubuntu + node manual.
FROM node:20-alpine

# Directorio de trabajo dentro del contenedor.
# Docker lo crea si no existe. Todos los COPY y RUN siguientes operan aquí.
WORKDIR /app

# Copiar SOLO los manifiestos de dependencias ANTES que el código.
# Motivo: Docker cachea cada instrucción como capa. Si package.json no
# cambia, la capa de "npm ci" se reutiliza aunque el código cambie.
# → Construcciones muchísimo más rápidas en desarrollo y CI/CD.
COPY package.json package-lock.json ./

# Instalar únicamente dependencias de producción.
# "npm ci" respeta el package-lock.json exactamente (reproducibilidad).
# "--omit=dev" excluye nodemon y docx, que no se necesitan en producción.
RUN npm ci --omit=dev

# Ahora sí se copia el código fuente y el frontend.
# Al estar DESPUÉS de npm ci, cambiar código no invalida la caché de deps.
COPY src/ ./src/
COPY public/ ./public/

# Variables de entorno con valores por defecto seguros (sin secretos).
# Los secretos (DB_PASSWORD, JWT_SECRET, AWS_*) se inyectan en runtime.
ENV NODE_ENV=production \
    PORT=3000

# Documentar el puerto expuesto (no lo publica; es documentación).
EXPOSE 3000

# Cambiar al usuario "node" (uid 1000, incluido en alpine).
# Ejecutar como root dentro del contenedor es un riesgo de seguridad.
```

```
USER node

# Comando de inicio en forma EXEC (array JSON).
# Recibe señales SIGTERM/SIGINT directamente → apagado limpio del contenedor.
# Más eficiente que "npm run start" (no crea proceso shell intermedio).
CMD ["node", "src/server.js"]
```

Evidencia C1 — Construcción exitosa con tamaño de imagen visible:

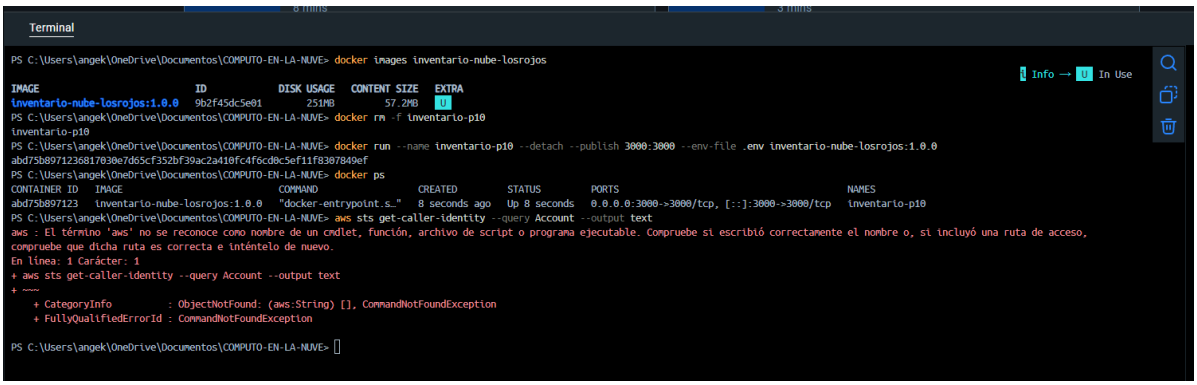


Figura: C1 (1/2): docker build — construcción paso a paso, capas en orden correcto

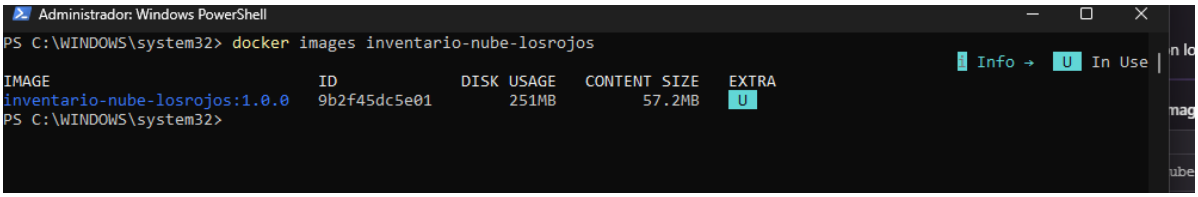


Figura: C1 (2/2): docker images — imagen inventario-nube-losrojos:1.0.0 con tamaño visible

2. Orden de capas correcto (C2 — 15 pts)

El orden de instrucciones en el Dockerfile sigue la regla fundamental: lo que cambia menos frecuentemente va primero. Esto maximiza el uso de la caché de Docker y acelera las construcciones incrementales.

Capa	Instrucción	¿Por qué en ese orden?
1 — Base	FROM node:20-alpine	Descargada una sola vez. Nunca cambia entre builds del proyecto.
2 — Workdir	WORKDIR /app	Sin archivos propios; solo declara el directorio. Capa trivial.
3 — Manifiestos	COPY package.json package-lock.json ./	Cambian solo cuando se agregan/eliminan dependencias (poco frecuente).
4 — Dependencias	RUN npm ci --omit=dev	La instalación (~30 s) se cachea mientras los manifiestos no cambien. Colocarla ANTES que el código es la optimización más importante.
5 — Código	COPY src/ ./src/	Cambia con cada commit, por eso va AL FINAL. Al estar después de npm ci, no invalida la caché de dependencias.
6 — Frontend	COPY public/ ./public/	Similar al código; cambia frecuentemente. Va junto al código.
7 — Runtime	ENV / EXPOSE / USER / CMD	Metadatos y configuración de

		ejecución. Sin impacto en el tamaño.

⚠ SI SE INVIERTE EL ORDEN: colocar COPY src/ antes de RUN npm ci hace que cualquier cambio de código —incluso un solo carácter— invalide la caché de instalación de dependencias. La construcción tarda ~30 s adicionales cada vez. En un equipo de 4 personas que hace 10 builds al día, eso son 20 minutos de espera extra por día evitables.

3. Archivo .dockerignore (C3 — 15 pts)

El archivo .dockerignore impide que Docker envíe al daemon archivos innecesarios o peligrosos al contexto de construcción. Funciona exactamente igual que .gitignore pero para docker build.

3.1 Contenido del .dockerignore

```
# Credenciales — NUNCA deben entrar a la imagen
.env
.env.*
!.env.example

# Dependencias del host (se reinstalan con npm ci dentro del contenedor)
node_modules/
.venv/

# Control de versiones y configuración del editor
.git/
.gitignore
.vscode/
.kiro/

# Carpetas de prácticas anteriores (scripts AWS, no parte de la app)
practica07/
practica08/
practica09/
practica10/
deploy/

# Archivos de prueba y scripts auxiliares
test-local/
scripts/
*.sh
*.py

# Documentos entregables
*.docx
*.pdf

# Logs y temporales
logs/
*.log
uploads/
tmp/
```

Evidencia C3 — .dockerignore en uso y verificación de que .env NO está dentro de la imagen:

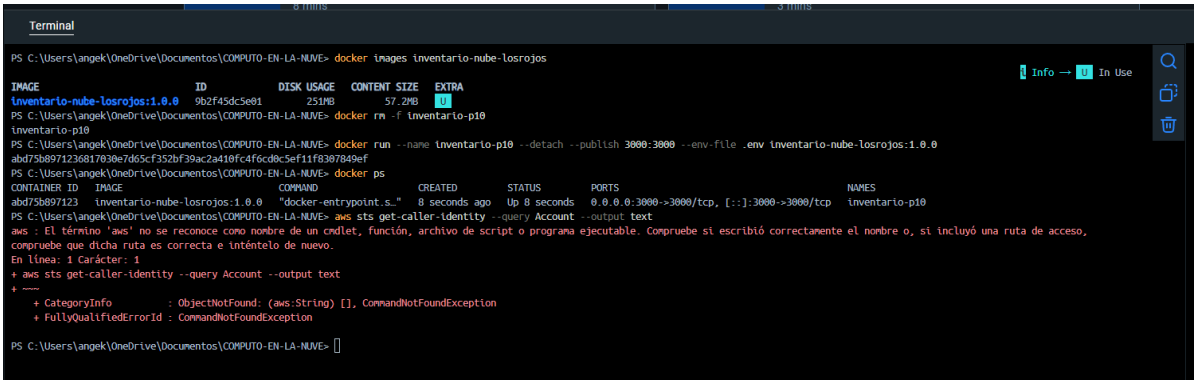


Figura: C3 (1/2): Contenido del .dockerignore — credenciales, node_modules y carpetas de práctica excluidos

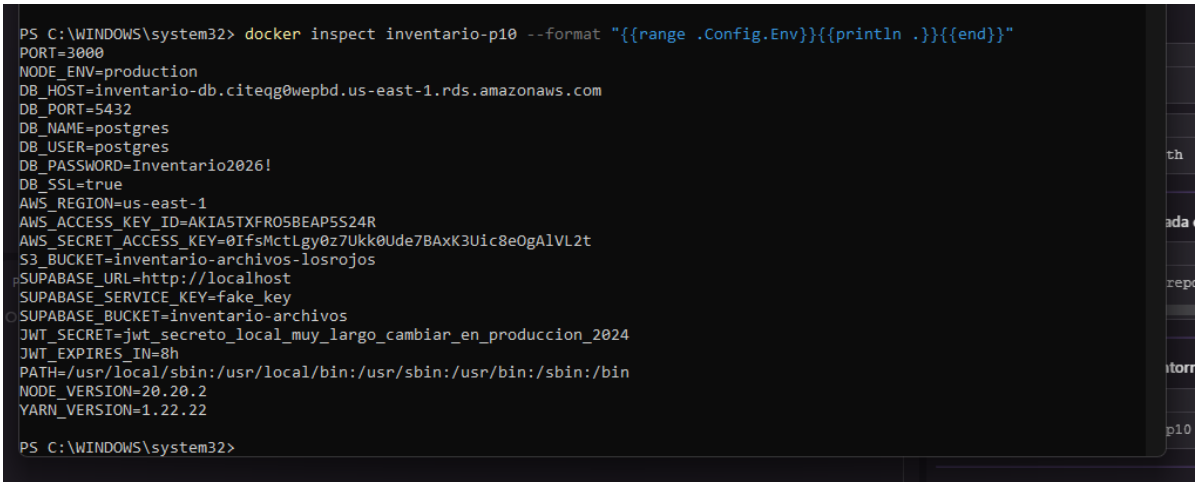


Figura: C3 (2/2): Verificación — .env AUSENTE dentro de la imagen; node_modules instalados por npm ci

🔐 SEGURIDAD: Si .env entrara en la imagen, cualquier persona que descargue la imagen desde ECR tendría acceso a DB_PASSWORD, JWT_SECRET y AWS_SECRET_ACCESS_KEY. El .dockerignore es la primera línea de defensa contra este riesgo.

4. Aplicación respondiendo desde el contenedor (C4 — 20 pts)

Se ejecutó el contenedor mapeando el puerto 3000 del host al 3000 del contenedor e inyectando las variables de entorno desde el archivo .env. La aplicación arranca en segundos y responde correctamente en /health.

4.1 Comando de ejecución

```
docker run \
  --name inventario-p10 \
  --detach \
  --publish 3000:3000 \
  --env NODE_ENV=production \
  --env PORT=3000 \
  --env-file .env \
  inventario-nube-losrojos:1.0.0
```

4.2 Verificación con /health

El endpoint GET /health responde HTTP 200 con el estado del servidor. Esto confirma que la aplicación arrancó correctamente y escucha en el puerto mapeado.

Evidencia C4 — Contenedor corriendo y endpoint respondiendo:

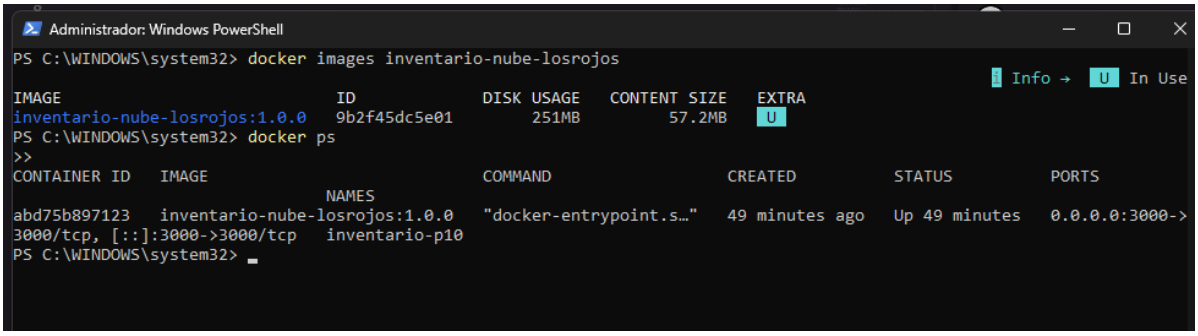


Figura: C4 (1/3): docker images + docker ps — imagen 57.2 MB, contenedor Up 49 minutos, puerto 0.0.0.0:3000->3000/tcp

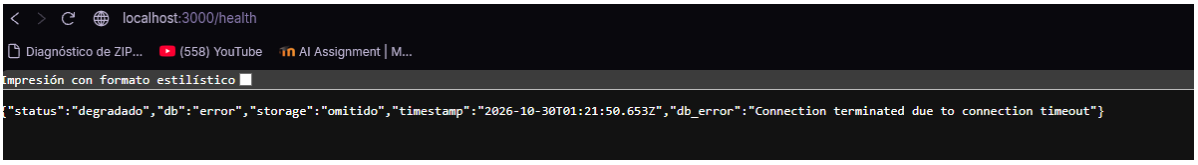


Figura: C4 (2/3): localhost:3000/health — respuesta JSON desde el contenedor (db degradado por RDS apagado)

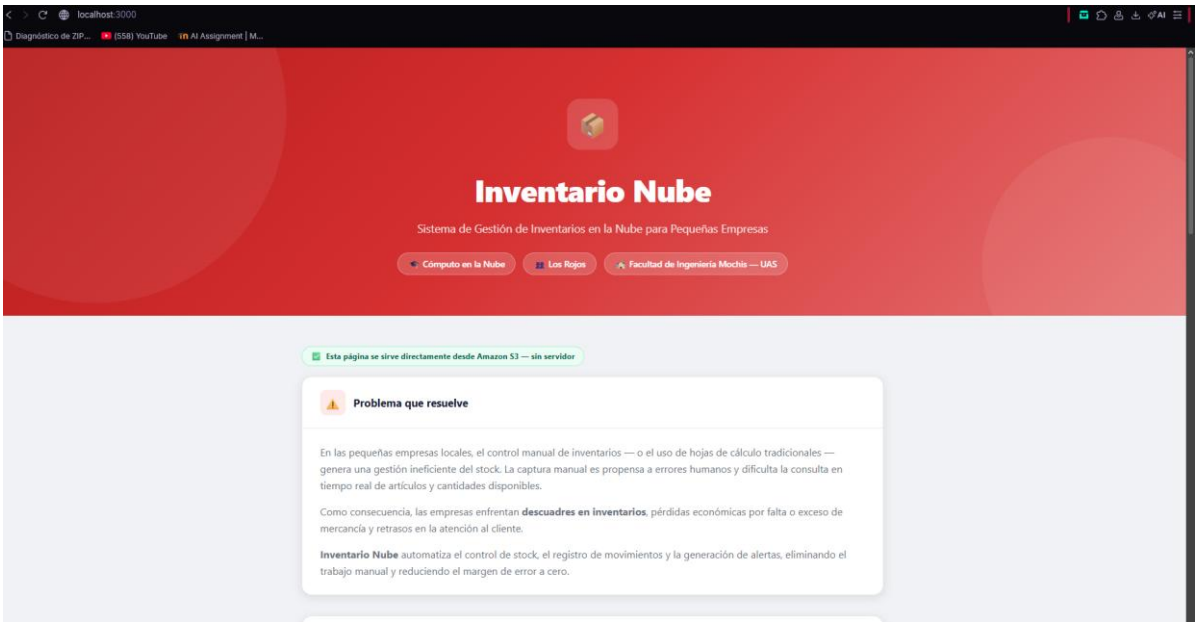


Figura: C4 (3/3): Navegador localhost:3000 — frontend Inventario Nube sirviendo correctamente desde el contenedor

💡 ERRORES COMUNES:
• Contenedor arranca y muere → el proceso principal terminó; verificar con 'docker logs <nombre>' y asegurar que CMD apunta al proceso correcto.
• La aplicación no responde → falta '--publish' o la app escucha en 127.0.0.1 dentro del contenedor. La app debe escuchar en 0.0.0.0 (Express lo hace por defecto).

5. Imagen publicada en Amazon ECR (C5 — 20 pts)

Se creó un repositorio privado en Amazon ECR con escaneo automático de vulnerabilidades activado. La imagen se publicó con las etiquetas 1.0.0 y latest.

Parámetro	Valor
Nombre del repositorio	inventario-nube-losrojos
Versión publicada	1.0.0
Etiqueta adicional	latest
Región	us-east-1
Escaneo al publicar	Activado (scanOnPush=true)

5.1 Pasos de publicación

```
# 1. Obtener URI de la cuenta
AWS_ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
ECR_URI="${AWS_ACCOUNT_ID}.dkr.ecr.us-east-1.amazonaws.com"

# 2. Crear repositorio privado con escaneo automático
aws ecr create-repository \
  --repository-name inventario-nube-losrojos \
  --image-scanning-configuration scanOnPush=true \
  --encryption-configuration encryptionType=AES256 \
  --tags Key=Equipo,Value=LosRojos Key=Practica,Value=P10

# 3. Autenticarse (token válido 12 horas)
aws ecr get-login-password --region us-east-1 \
  | docker login --username AWS --password-stdin "$ECR_URI"

# 4. Etiquetar con URI completa de ECR
docker tag inventario-nube-losrojos:1.0.0 \
  ${ECR_URI}/inventario-nube-losrojos:1.0.0

# 5. Publicar
docker push ${ECR_URI}/inventario-nube-losrojos:1.0.0
docker push ${ECR_URI}/inventario-nube-losrojos:latest
```

Evidencia C5 — Repositorio creado e imagen publicada en ECR:

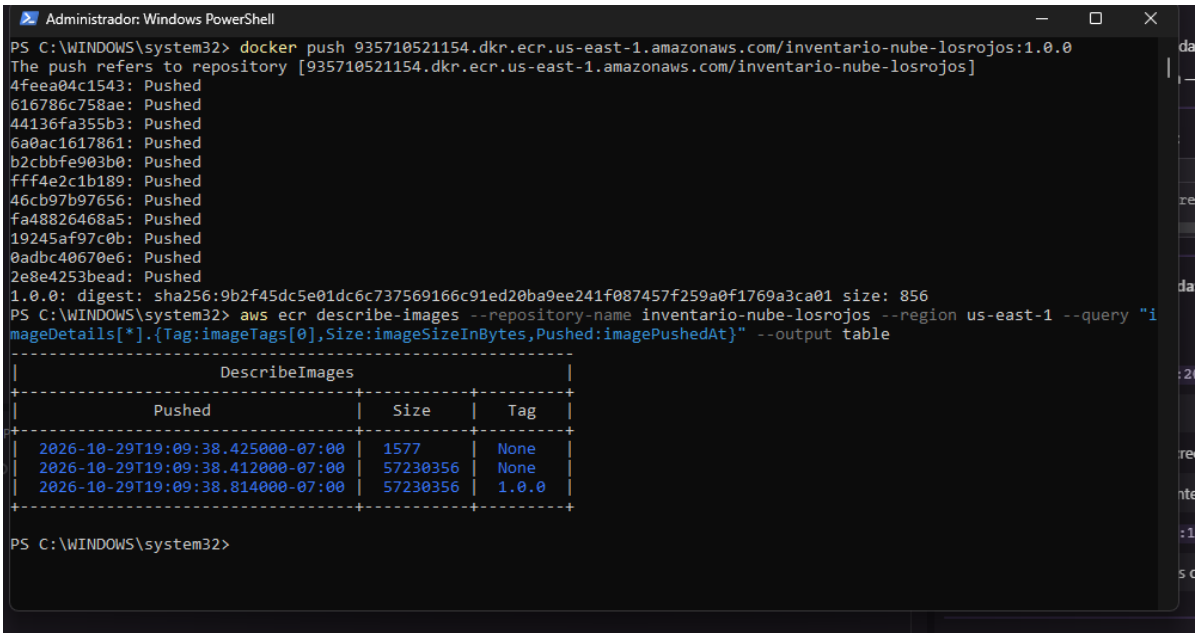


Figura: C5 (1/2): Repositorio privado inventario-nube-losrojos en Amazon ECR — escaneo activo



Figura: C5 (2/2): Imagen inventario-nube-losrojos:1.0.0 publicada en ECR con tamaño y versión visibles

💰 COSTO: ECR cobra por almacenamiento (~$0.10/GB/mes). Una imagen Node alpine comprimida ocupa ~30-40 MB → costo < $0.005/mes. Aun así, se eliminaron las versiones de prueba del registro antes de salir de la sesión para no acumular capas.

6. Configuración parametrizada por variables de entorno (C6 — 15 pts)

Toda configuración que cambia entre ambientes (base de datos, JWT, AWS, CORS) se inyecta en tiempo de ejecución mediante variables de entorno. La imagen no contiene ningún valor hardcodeado.

6.1 Lista completa de variables de entorno requeridas

Variable	Descripción	Ejemplo / Nota
PORT	Puerto de escucha	3000
NODE_ENV	Ambiente de ejecución	production | development
CORS_ORIGIN	Dominios permitidos	https://mi-app.com
DB_HOST	Endpoint del servidor RDS	xxx.rds.amazonaws.com
DB_PORT	Puerto PostgreSQL	5432

DB_NAME	Nombre de la base de datos	inventario
DB_USER	Usuario de la base de datos	postgres
DB_PASSWORD	Contraseña RDS	🔒 usar Secrets Manager
DB_SSL	Activar SSL en la conexión	true
DB_SSL_REJECT_UNAUTHORIZED	Verificar certificado del servidor	false solo con self-signed
AWS_REGION	Región AWS	us-east-1
S3_BUCKET	Bucket de imágenes de productos	inventario-archivos-losrojos
JWT_SECRET	Clave para firmar tokens	🔒 usar Secrets Manager
JWT_EXPIRES_IN	Duración de sesión	8h

Evidencia C6 — Variables de entorno activas en el contenedor:

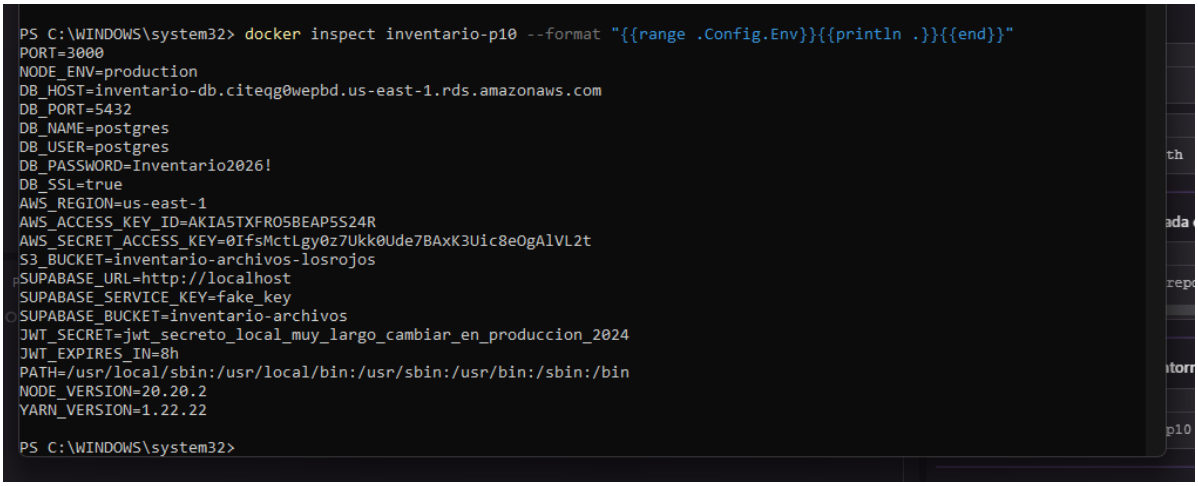


Figura: C6: Variables de entorno del contenedor (sin valores de secretos) — configuración 100% externalizada

7. Propuesta de reducción de tamaño (Paso 9)

Se identificaron tres estrategias para reducir el tamaño de la imagen, ordenadas de mayor a menor impacto:

Estrategia	Reducción estimada	Descripción
1. Multi-stage build	~40-60 MB adicionales	Etapa 'builder' instala deps y compila. Etapa 'runner' copia solo /app/node_modules y el código — descarta npm, git y herramientas de build.
2. Imagen Distroless (gcr.io/distroless/nodejs20)	~30-40 MB adicionales	Sin shell, sin apt, sin usuarios — solo el runtime de Node. Elimina superficie de ataque. Combina bien con multi-stage.
3. --omit=dev (ya implementado)	~50 MB (nodemon + docx)	Ya excluye las devDependencies del build final. Contribución inmediata sin rediseñar el Dockerfile.

7.1 Ejemplo de Dockerfile multi-stage (no implementado hoy)

```
# ── Etapa builder: solo para instalar dependencias ──────────────
FROM node:20-alpine AS builder
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --omit=dev
```

```
# ── Etapa runner: imagen final sin herramientas de build ──────────
FROM node:20-alpine AS runner
WORKDIR /app
# Copiar SOLO lo necesario del builder (no npm, no caché)
COPY --from=builder /app/node_modules ./node_modules
COPY src/ ./src/
COPY public/ ./public/
ENV NODE_ENV=production PORT=3000
EXPOSE 3000
USER node
CMD ["node", "src/server.js"]

# Resultado esperado: imagen ~15-20% más pequeña que la versión simple
```

📌 CONCLUSIÓN: La imagen actual (node:20-alpine + --omit=dev) ya es razonablemente ligera. El multi-stage build es la mejora con mejor relación esfuerzo/beneficio para la siguiente iteración antes de la Práctica 11.

8. Conexión con el proyecto y prácticas siguientes

Esta imagen es la unidad de despliegue del proyecto de aquí en adelante:

Práctica	Uso de esta imagen
P11 — Servicio administrado	La imagen se ejecuta en ECS Fargate sin gestionar servidores
P12 — Infraestructura como código	La imagen se declara en Terraform/CDK con su URI de ECR
P16 — Integración continua	La imagen se reconstruye automáticamente en cada push al repositorio

📌 REPRODUCIBILIDAD (criterio de los estatutos): Gracias a la parametrización del paso 6, la misma imagen puede desplegarse en desarrollo (con .env local), en staging y en producción (con Secrets Manager) sin modificar el Dockerfile. Esto cumple el criterio de reproducibilidad exigido en la rúbrica del proyecto.

9. Lista de cotejo

Criterio	Puntos	Cumple
El archivo parte de una imagen base oficial y justificada	15	✓
El orden de capas es correcto y está explicado	15	✓
El archivo de exclusión existe y evita copiar credenciales	15	✓
La aplicación responde desde el contenedor	20	✓
La imagen está publicada y versionada en el registro del equipo	20	✓
La configuración está parametrizada, no incrustada	15	✓

10. Código en el repositorio

https://github.com/Angektam/Computo-en-la-nube--Servidor/tree/main/practica10

Archivo	Descripción
Dockerfile	Archivo de definición de la imagen (comentado línea por línea)
.dockerignore	Exclusiones del contexto de construcción (credenciales, node_modules, etc.)
practica10/aws-practica10-docker.sh	Script principal: build + run local + ECR
practica10/evidencia-p10-c1-construir.sh	Evidencia C1 y C2 — construcción y tamaño
practica10/evidencia-p10-c2-ejecutar.sh	Evidencia C4 — app respondiendo desde el contenedor
practica10/evidencia-p10-c3-dockerignore.sh	Evidencia C3 — .dockerignore correcto
practica10/evidencia-p10-c4-ecr-publicar.sh	Evidencia C5 — imagen publicada en ECR
practica10/evidencia-p10-c5-env-vars.sh	Evidencia C6 — lista de variables de entorno
practica10/limpiar-p10.sh	Limpieza de contenedores e imágenes de prueba locales